



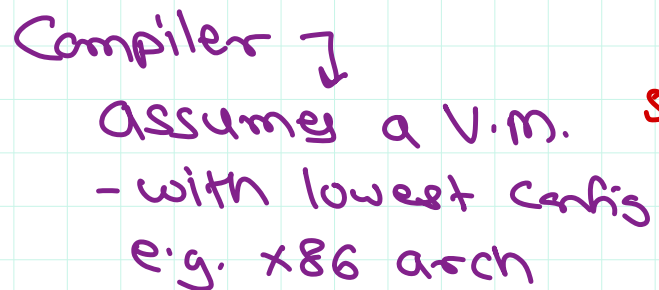
Embedded Operating Systems

Trainer: Nilesh Ghule



- ① general intro
- ② process mgmt
- ③ file & io mgmt
- ④ memory mgmt

• $\mathbb{R} \times \mathbb{R}$



80386
(4GB)

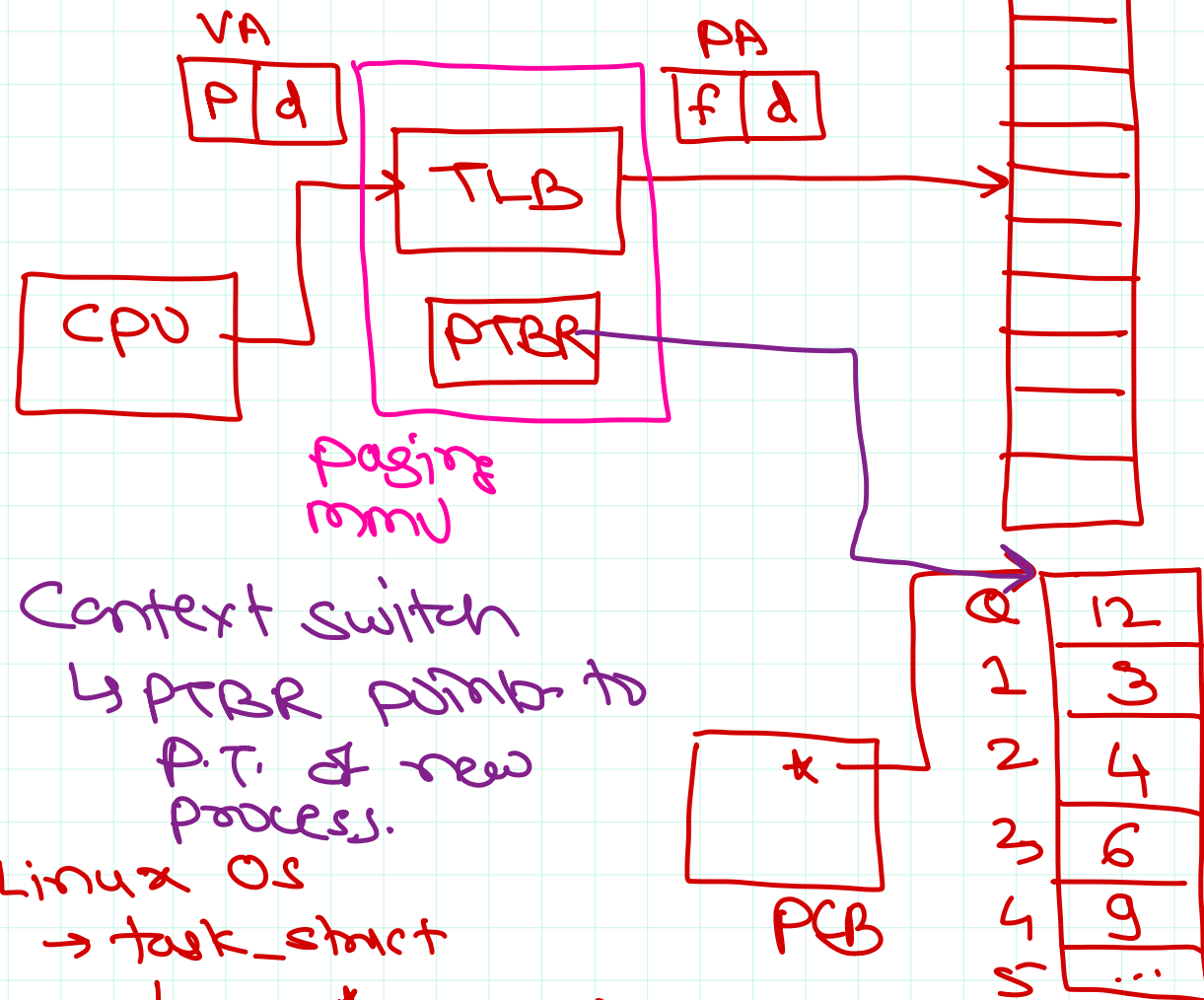
objdump -S program.exe → instr addr
objdump -t program.exe → var addr

Paging

RAM → fourmes (4K or 4m)

process $\rightarrow$ pages

1 page $\rightarrow$ frame 1



Context switch
↳ PTBR points to
P.T. of new
process.

Linux OS

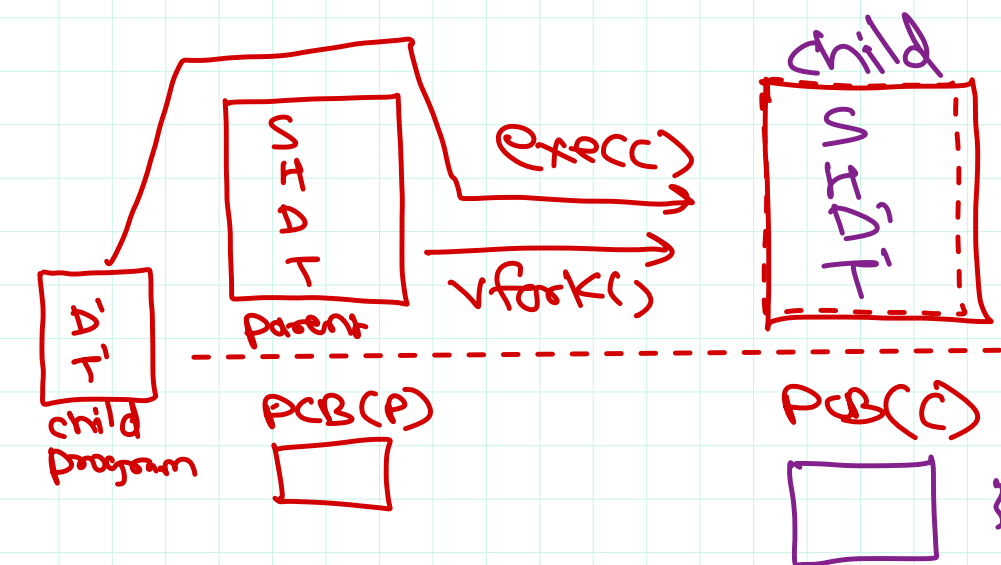
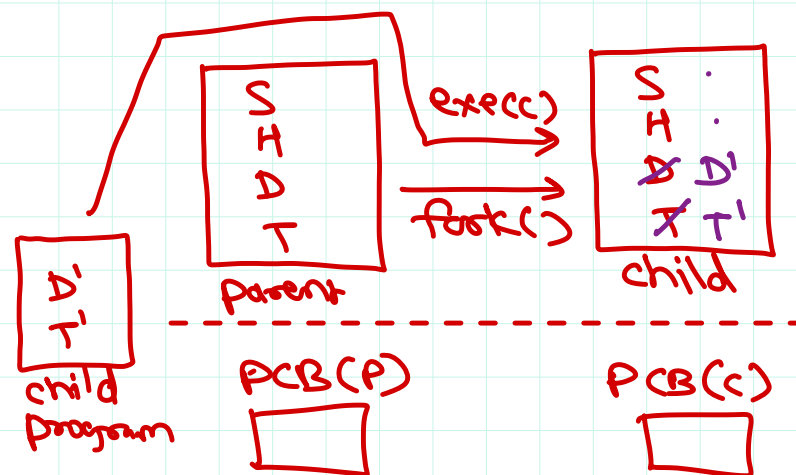
→ task_struct

↳ $mm^k \rightarrow mm_struct$

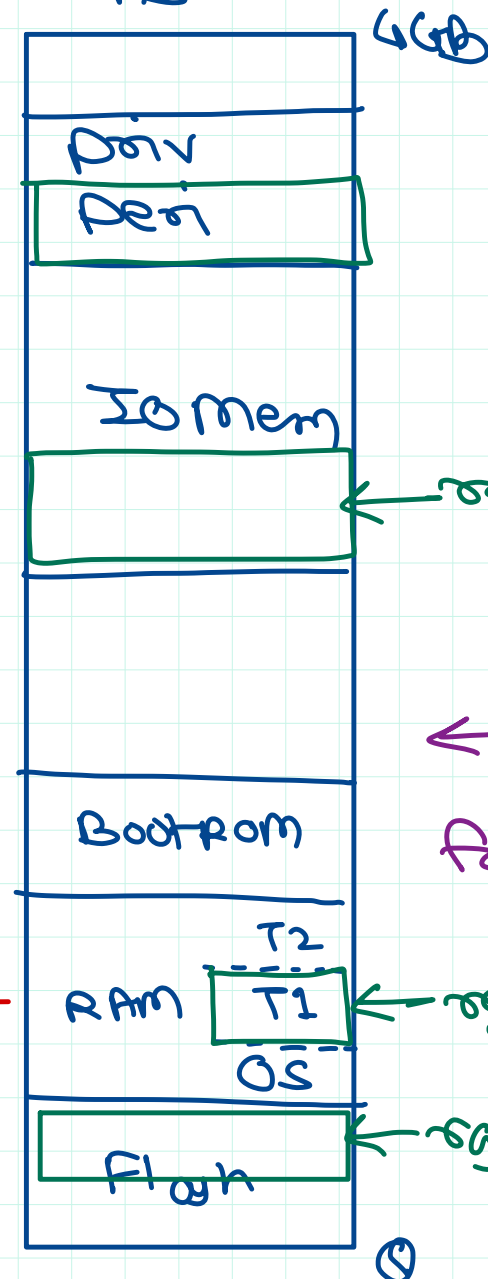
↳ $*pgd \rightarrow pgtable$

P.T.
Nilesh Ghule

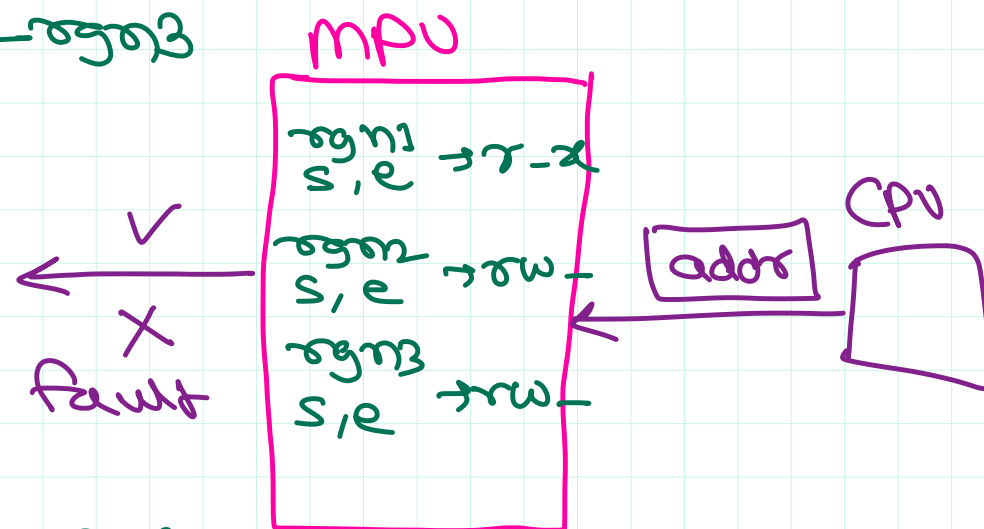
fork() vs vfork() (BSD UNIX)



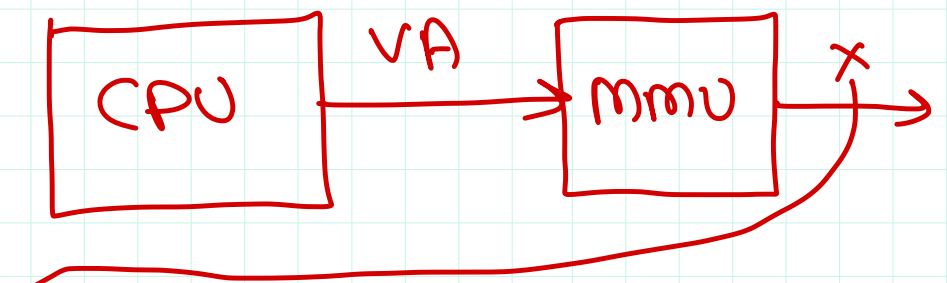
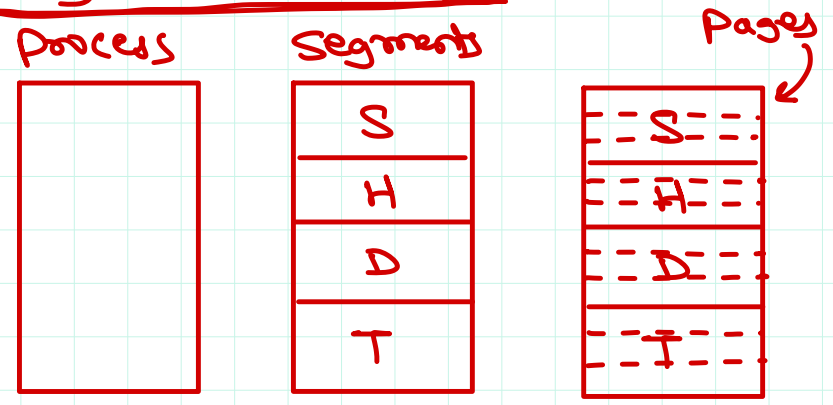
Processor AS



- defines mem sections & their perm. (rwx)
- ① start addr
 - ② end addr
 - ③ permissions



Segmented Paging

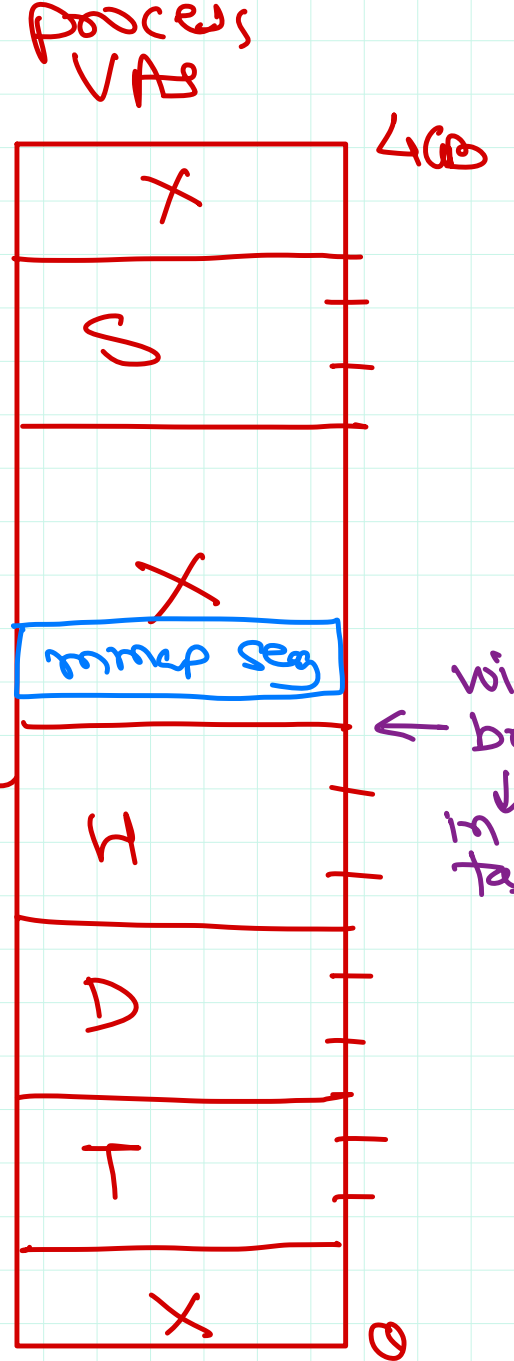


Page Fault :

- Why →
- ❌ ① VA is unmapped
 - ❌ ② VA invalid perm
 - ✓ ③ VA page swapped
 - ✓ ④ VA Page unalloc

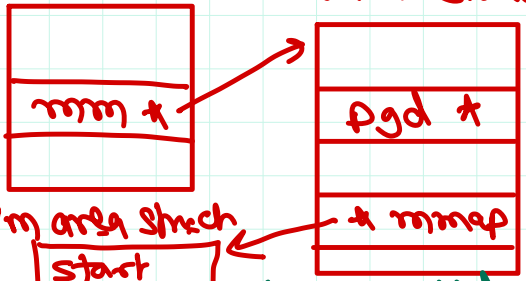
Page Fault handler

- ① check if v addr is legal & perm is valid.

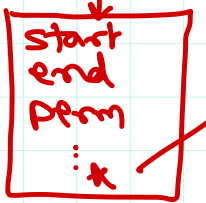
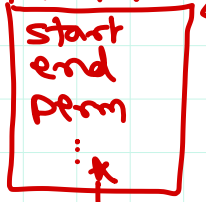


Linux keep seg info in VAs.
(Vm_area_struct).

task_struct → mm_struct

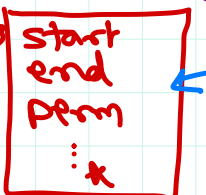


vm area struct

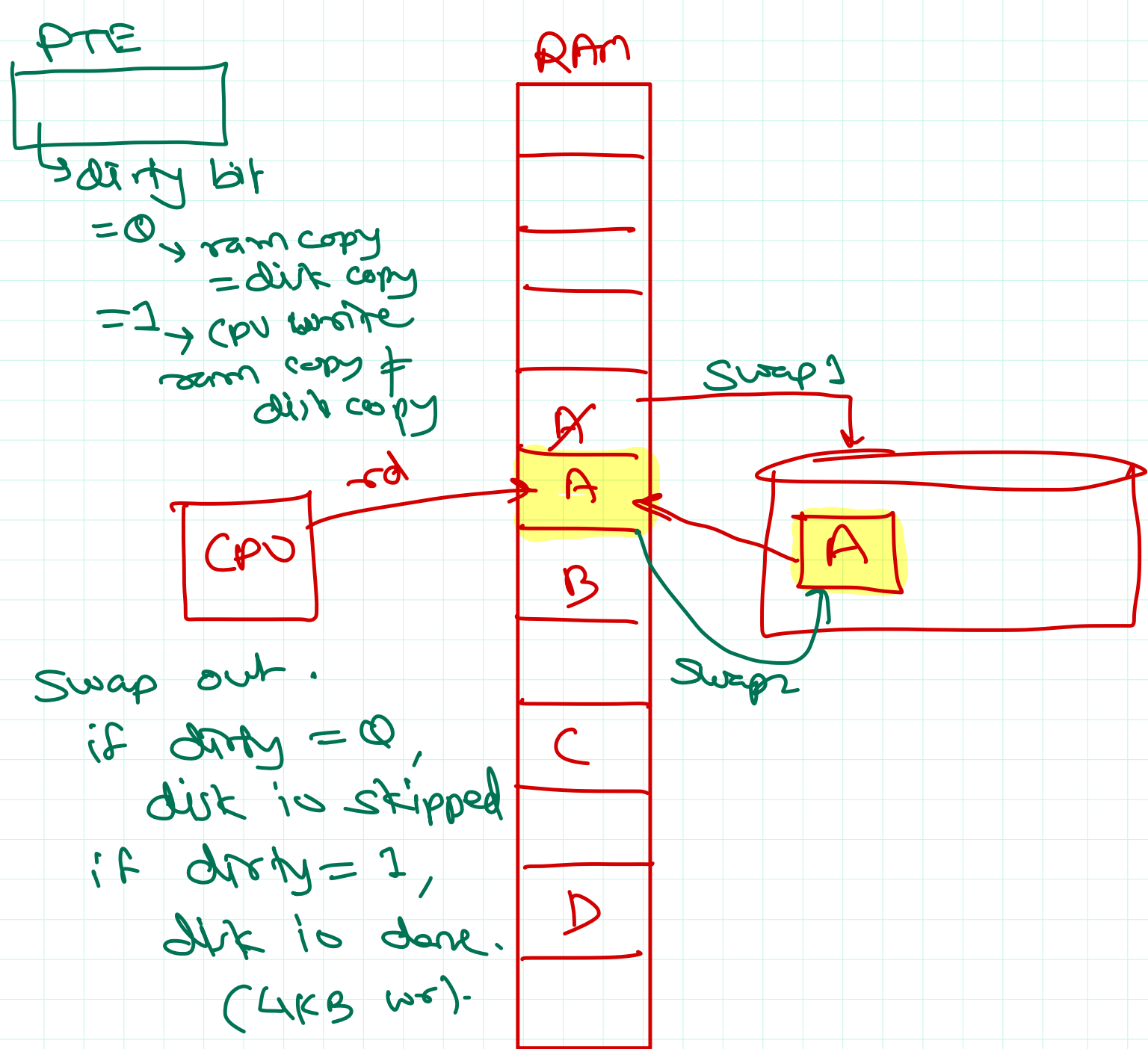


checked in seg info (Vm_area_struct) ?
if invalid generate SIGSEGV
Signal → default sig handler
"Segmentation Fault!"

cat /proc/pid/maps



mmap() syscall → create new seg.
- vm-area_struct
+ Page table entries (initially invalid)

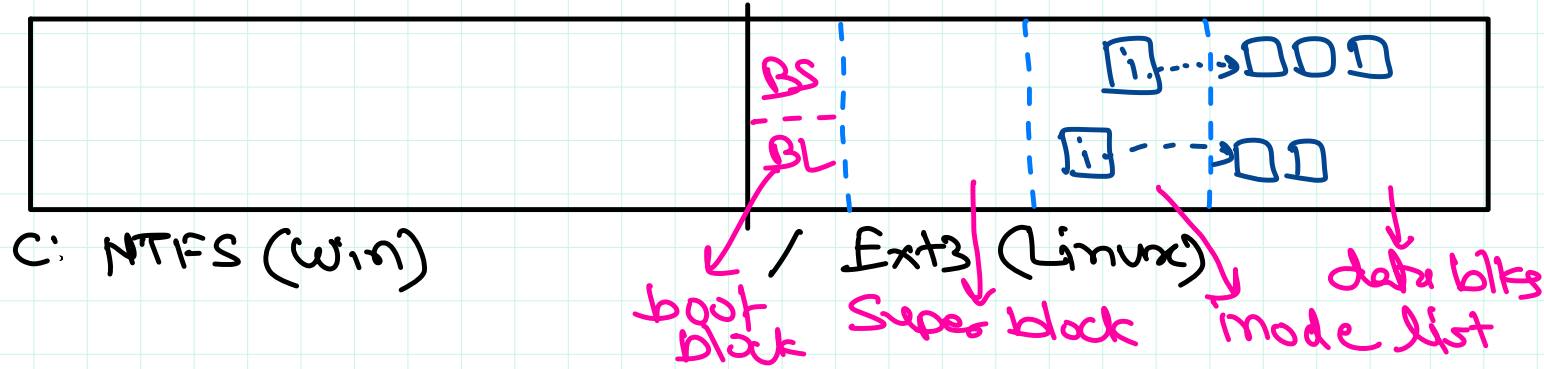


Swap out.

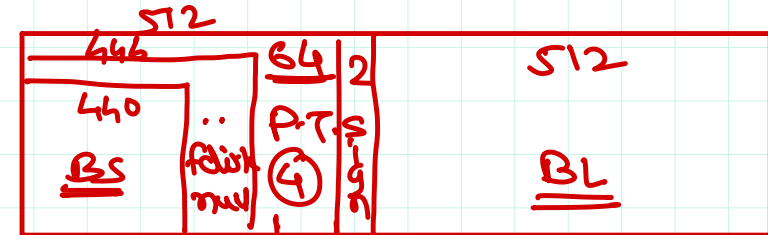
if dirty = 0,
disk is skipped

if dirty = 1,
disk is done.
(4KB wr).

File System → Organization of files on disk.



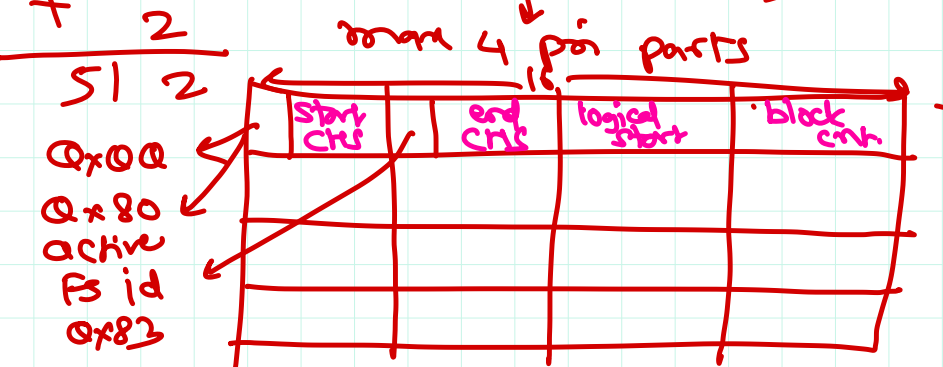
File = Data (blocks) + metadata (inode)



1024

64 bytes → 4 entries (MBR partitioning)

more 4 par parts



Semaphore

- ① counter
 $\rightarrow inc \rightarrow V$
 $\rightarrow dec \rightarrow P$
- ② applies
 $\rightarrow$ counting
 $\rightarrow$ mutual excl
 $S = 1$
 $\rightarrow$ Cond Flag
 $S = 0$
- ③ Process 1 $\rightarrow dec$
 Process 2 $\rightarrow inc$ ✓
- ④ User space & kernel space.
- ⑤ $P()$ op on sema with $cnt = 0$, block the process.

Mutex

- ① lock / flag
 $\rightarrow busy = 1$
 $\rightarrow free = 0$
- ② mutual exclusion
- ③ only locking process can unlock.
- ④ User space & kernel space.
- ⑤ locking already locked mutex $\rightarrow$ block process.

Spinlock

- ① lock / flag
 $\rightarrow busy = 1$
 $\rightarrow free = 0$
- ② mutual exclusion
- ③ only locking process can unlock.
- ④ kernel space.
- ⑤ locking already locked spinlock $\rightarrow$ next process will busy wait (loop) - running state ... until lock is released.

hw based sync mech.
 $\rightarrow$ bus lock train
 $\rightarrow$ SWP
 $\rightarrow$ LDREX, STREX

⑥ Used when sleep not allowed
 e.g. TCP half.

Dead lock

In multi-threading sync programs, multiple threads waiting for sync are permanently blocked → dead lock.

Deadlock characteristics,

- ① no preemption
- ② mutual exclusion
- ③ hold & wait
- ④ circular wait

Deadlock vs Starvation.

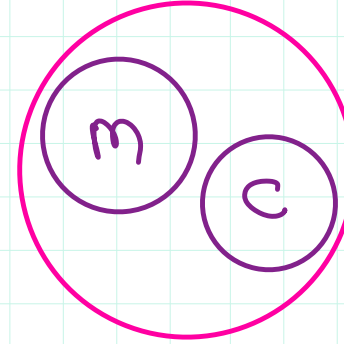
DL → blocked processes are in waiting state → wait que.

— wait for resources

Starvation → processes waiting in ready state → ready queue

— not getting CPU due to low priority

Monitor → sync mech in few systems like Java



— can be created in Linux = mutex + cond var.

— java monitor
Synchronized (obj) → attached to monitor
→ lock mutex

- ① unlock mutex ← obj.wait();
- ② sleep current process on cond var. → unlock mutex
- ③ lock mutex again & proceed. ← other thread obj.notify()



Thank you!

Nilesh Ghule <nilesh@sunbeaminfo.com>

